

Below is the revised requirements document, scoped specifically to a **static, browser-based MLB newspaper generated several times per day** and published to:

```
https://sportzballz.io/daily-sports-page/index.html
```

 Edit



Daily Sports Page

Static HTML Generation Requirements

Product: SportzBallz Daily Sports Page

Primary URL: `https://sportzballz.io/daily-sports-page/index.html`

Version: 1.0

Status: Implementation Requirements

Primary Sport: Major League Baseball

Delivery Model: Pre-generated static HTML

Client Platforms: Desktop, tablet, and mobile web browsers

1. Product Summary

The Daily Sports Page is a browser-based, newspaper-style MLB publication generated several times each day.

The product should resemble a traditional printed sports section rather than a modern sports application. It should combine current MLB scores, schedules, standings, league leaders, game summaries, analysis, and editorial content into one self-contained static HTML page.

The page will be generated by an automated publishing pipeline and written to:

```
/daily-sports-page/index.html
```

The public URL will be:

```
https://sportzballz.io/daily-sports-page/index.html
```

There will be:

- No native mobile application
- No React Native or Flutter application
- No application server required to render the page
- No client-side dependency on sports APIs
- No user account system
- No login requirement
- No database dependency for page viewing
- No infinite scrolling news feed
- No runtime AI generation in the browser

All sports data, written content, and page structure must be generated before publication.

2. Product Vision

The Daily Sports Page should recreate the experience of opening a traditional newspaper sports section while using current MLB data and automated editorial generation.

The page should feel:

- Authoritative
- Calm
- Dense but readable
- Nostalgic
- Statistical
- Editorially curated
- Fast-loading
- Printable
- Distinct from modern card-based sports sites

The primary product principle is:

Deliver the most useful parts of the traditional baseball sports page in a current, automatically generated static web edition.

3. Core User Experience

A visitor opens the page and immediately sees the latest published MLB edition.

The visitor should be able to:

1. Read the lead MLB story.
2. Review completed game scores.
3. See games currently in progress.
4. Review upcoming games.
5. inspect probable pitchers and game details.
6. View division and wild-card standings.
7. View top 10 league leaders in major categories.
8. Read short game recaps and league-wide analysis.
9. Review injuries, transactions, and notable developments.
10. Print or save the page as a PDF.
11. Share links to sections or stories.
12. Understand when the edition was generated and how current its data is.

The visitor should not need to create an account or interact with a complex application shell.

4. Publishing Frequency

The page will be regenerated several times per day.

The generation schedule must be configurable rather than hard-coded.

Recommended initial schedule in `America/New_York`:

```
6:00 AM    Morning edition
12:00 PM   Midday update
5:00 PM    Evening preview
11:30 PM   Late edition
```

Additional generation runs may occur:

- After a major game concludes
- After a significant transaction
- Before the first game of the day
- After the final MLB game concludes

- When source data is corrected
- When the publisher manually triggers a rebuild

Each successful generation replaces the currently published

`index.html` .

The system should optionally preserve dated editions separately.

Example:

```
/daily-sports-page/index.html  
/daily-sports-page/archive/2026-07-13-0600.html  
/daily-sports-page/archive/2026-07-13-1200.html  
/daily-sports-page/archive/2026-07-13-1700.html  
/daily-sports-page/archive/2026-07-13-2330.html
```

The archive is optional for the first release but the generator architecture should support it.

5. Static-Site Architecture

The final published page must be static HTML.

The browser must not contact sports-data APIs to construct the page.

5.1 Generation Flow

```
Sports data providers  
    ↓  
Data collection  
    ↓  
Data validation and normalization  
    ↓  
Derived statistics and editorial selection  
    ↓  
AI-assisted story generation  
    ↓  
Content validation  
    ↓  
Edition JSON  
    ↓  
Static HTML renderer
```

↓
HTML validation
↓
Atomic publication
↓
CDN or web-server delivery

5.2 Generated Output

At minimum, each publication run should create:

`index.html`

The preferred production output is:

`index.html`
`assets/daily-sports-page.css`
`assets/daily-sports-page.js`
`assets/images/*`
`data/edition.json`

A fully self-contained single HTML file is also acceptable for the MVP.

If external assets are used, they must be:

- Versioned or cache-busted
- Hosted on the same domain where practical
- Available before `index.html` is published
- Referenced using stable relative paths

5.3 Static Rendering Rules

All essential content must exist in the generated HTML.

JavaScript may enhance the experience, but it must not be required to read the core page.

The following must remain usable when JavaScript is disabled:

- Main headline
- Lead story
- Scores
- Schedule
- Standings
- League leaders

- Game recaps
- Transactions
- Injuries
- Publication timestamp

JavaScript may be used for:

- Switching league-leader categories
 - Collapsing secondary sections
 - Print controls
 - Theme switching
 - Page navigation
 - Client-side filtering
 - Copy-link buttons
-

6. Page Structure

The page should use semantic HTML and a traditional newspaper hierarchy.

Recommended structure:

```
<body>
  <header></header>
  <nav></nav>
  <main>
    <section id="front-page"></section>
    <section id="scoreboard"></section>
    <section id="todays-games"></section>
    <section id="standings"></section>
    <section id="league-leaders"></section>
    <section id="game-recaps"></section>
    <section id="around-the-league"></section>
    <section id="transactions"></section>
    <section id="injuries"></section>
    <section id="history"></section>
  </main>
  <footer></footer>
</body>
```

Every major section must have a stable anchor ID.

Examples:

```
#scoreboard
#todays-games
#standings
#league-leaders
#game-recaps
#transactions
#injuries
```

These anchors will support navigation and direct sharing.

7. Masthead and Edition Metadata

The top of the page must include:

- Publication name
- Edition type
- Publication date
- Generation timestamp
- Time zone
- Data freshness statement
- Navigation links
- Optional weather or schedule summary

Example:

```
THE DAILY SPORTING PRESS
Major League Baseball Edition
```

```
Monday, July 13, 2026
Morning Edition
Generated at 6:02 AM ET
Scores and statistics current through 5:55 AM ET
```

The publication time and data cutoff time must be separate fields.

This distinction is important because data collection may complete before HTML generation.

8. Edition Types

The generator should assign an edition type based on generation time and available data.

Supported types:

- Morning Edition
- Midday Edition
- Evening Edition
- Late Edition
- Final Edition
- Special Edition

The edition label should influence content selection.

8.1 Morning Edition

Emphasize:

- Previous day's results
- Full game recaps
- Updated standings
- League leaders
- Today's schedule
- Probable pitchers
- Morning analysis

8.2 Midday Edition

Emphasize:

- Lineup announcements
- Weather changes
- Injuries
- Transactions
- Day games
- Updated probable pitchers

8.3 Evening Edition

Emphasize:

- Night-game previews
- Confirmed lineups
- Starting pitchers

- Weather
- Key matchups
- Games already completed

8.4 Late or Final Edition

Emphasize:

- Final scores
 - Completed box scores
 - Updated standings
 - Player-of-the-day selections
 - Late game recaps
 - Remaining games in progress
-

9. Front Page Requirements

The front page must provide a league-wide MLB view.

It must not be tailored primarily to Philadelphia or any other team.

9.1 Lead Story

The lead story should represent the most significant current MLB development.

Potential selection factors:

- Playoff implications
- Historic performance
- No-hitter or perfect-game attempt
- Major injury
- Major trade
- Division race
- Record-setting performance
- Rivalry series
- Exceptional individual performance
- League-wide strategic significance

Required lead story fields:

```
{  
  "headline": "",
```

```
"deck": "",
"byline": "",
"body": [],
"primaryImage": null,
"caption": "",
"sourceDataReferences": [],
"storyType": "",
"teams": [],
"players": []
}
```

9.2 Secondary Stories

The front page should contain three to six secondary stories.

Suggested categories:

- Game of the day
 - Player of the day
 - Pennant-race update
 - Rookie watch
 - Trade watch
 - Injury development
 - Statistical milestone
 - Controversial call or unusual event
-

10. Scoreboard

The scoreboard must include all MLB games relevant to the current edition.

Games must be grouped by state:

- Final
- In progress
- Delayed
- Postponed
- Suspended
- Scheduled

Each game should include:

- Away team

- Home team
- Away score
- Home score
- Game status
- Inning
- Outs, when available
- Winning pitcher
- Losing pitcher
- Save
- Start time
- Ballpark
- Doubleheader designation
- Postponement or delay reason
- Link or anchor to the game recap

Example:

FINAL

Los Angeles 6 10 0

San Diego 4 8 1

WP: Glasnow

LP: Cease

SV: Phillips

The scoreboard should use compact newspaper-style formatting rather than large visual cards.

11. Today's Games

The schedule section must list every MLB game scheduled for the edition date.

Required information:

- Game time in Eastern Time
- Away team
- Home team
- Probable starting pitchers
- Pitcher handedness
- Pitcher record

- Pitcher ERA
- Ballpark
- Television or streaming information, when legally available
- Weather summary
- Delay or postponement risk
- Doubleheader information
- Game status
- Optional betting information

The page must clearly distinguish confirmed and probable information.

Examples:

Probable pitcher

Confirmed starter

Lineup pending

Weather risk

Game postponed

12. Standings

The standings section must provide full MLB standings.

Required groupings:

- AL East
- AL Central
- AL West
- NL East
- NL Central
- NL West
- American League wild card
- National League wild card

Required columns:

- Team
- Wins
- Losses
- Winning percentage
- Games behind

- Wild-card games behind, where applicable
- Last 10
- Current streak
- Home record
- Away record
- Run differential

On smaller screens, secondary columns may be hidden or moved into a horizontally scrollable table.

The core columns must always remain visible:

- Team
 - W
 - L
 - PCT
 - GB
-

13. League Leaders

The league-leaders section must show the top 10 qualified MLB players in multiple categories.

The section must cover the entire MLB, not a selected team.

13.1 Batting Categories

Initial required categories:

- Batting average
- On-base percentage
- Slugging percentage
- OPS
- Home runs
- RBI
- Runs
- Hits
- Doubles
- Triples
- Stolen bases
- Walks

- Strikeouts
- Total bases
- Extra-base hits
- WAR, when data rights permit

13.2 Pitching Categories

Initial required categories:

- ERA
- Wins
- Strikeouts
- WHIP
- Saves
- Holds, when available
- Innings pitched
- Quality starts
- Complete games
- Shutouts
- Opponent batting average
- K/9
- BB/9
- HR/9
- Pitching WAR, when data rights permit

13.3 Leader Table Fields

Each row should contain:

- Rank
- Player
- Team
- Position or pitcher role
- Statistic value
- Games played or innings pitched
- League designation
- Qualification indicator

Example:

HOME RUNS

RK	PLAYER	TEAM	HR
----	--------	------	----

1	Aaron Judge	NY Yankees	34
2	Shohei Ohtani	LAD	31
3	Kyle Schwarber	PHI	29

13.4 Category Interaction

The HTML may display leader categories using:

- Tabs
- Select control
- Multiple newspaper columns
- Expandable sections

The default visible categories should be:

- Home runs
- RBI
- Batting average
- OPS
- ERA
- Strikeouts
- Wins
- Saves

All leader data must already be embedded in the static page or an accompanying static JSON file.

No live API call may be required when a user changes categories.

14. Game Recaps

Each completed game should have a short recap.

Minimum recap requirements:

- Headline
- Final score
- One- to three-paragraph summary
- Winning pitcher
- Losing pitcher
- Save
- Key offensive performer

- Key turning point
- Home runs
- Notable milestones
- Attendance, when available
- Time of game, when available

Recaps should be ordered by editorial significance rather than only by start time.

The generator should identify:

- Largest comeback
- Closest game
- Biggest upset
- Best pitching performance
- Highest scoring game
- Most important standings result
- Walk-off game
- Extra-inning game

These labels may appear as small newspaper-style tags.

15. Box Scores

Full box scores are desirable but may be deferred after the MVP.

When included, they should provide:

- Runs by inning
- Runs, hits, and errors
- Batting lines
- Pitching lines
- Home runs
- Doubles and triples
- Stolen bases
- Errors
- Left on base
- Double plays
- Winning pitcher
- Losing pitcher
- Save

- Umpires
- Attendance
- Time of game
- Weather
- Venue

To avoid an excessively long front page, full box scores may be placed in:

```
/daily-sports-page/games/{game-id}.html
```

The main `index.html` may link to these static game pages.

For the first release, compact box scores embedded in `index.html` are sufficient.

16. Around the League

This section should provide league-wide editorial summaries.

Potential recurring items:

- Three things we learned
- Division-race snapshot
- Wild-card watch
- Bullpen report
- Rookie watch
- Injury watch
- Trade watch
- Hot hitters
- Cold hitters
- Best pitching performances
- Series to watch
- Statistical oddities
- Milestones approaching

Each item should be concise enough to support a newspaper-column layout.

17. Transactions

The transactions section should include current MLB roster activity.

Transaction types:

- Trade
- Signing
- Call-up
- Option
- Designation for assignment
- Release
- Injured-list placement
- Injured-list activation
- Suspension
- Waiver claim
- Retirement

Each item should include:

- Team
- Player
- Transaction type
- Effective date
- Short explanation
- Source timestamp

Duplicate transactions from multiple providers must be deduplicated.

18. Injury Report

The injury section should contain significant injury updates.

Fields:

- Player
- Team
- Injury
- Roster status
- Date of injury
- Expected return, when reliably available
- Latest update
- Update timestamp

- Confidence level

The system must not invent a return date.

When no verified return date exists, use language such as:

No timetable announced

Day-to-day

Expected to be reevaluated

Return date not confirmed

19. Historical Content

The page may include a small historical baseball section.

Possible content:

- This day in baseball history
- Player birthdays
- Historic game anniversary
- Record anniversary
- Debut anniversary
- World Series anniversary
- Notable milestone

Historical claims must come from verified structured data or trusted references.

This section must not delay publication if historical data is unavailable.

20. Data Model

Every edition should first be represented as structured data before HTML rendering.

Recommended root schema:

```
{
  "edition": {
    "id": "2026-07-13-0600",
    "type": "morning",
```

```
    "date": "2026-07-13",
    "generatedAt": "2026-07-13T06:02:14-04:00",
    "dataCurrentThrough": "2026-07-13T05:55:00-04:00",
    "timezone": "America/New_York",
    "status": "published"
  },
  "leadStory": {},
  "secondaryStories": [],
  "games": [],
  "standings": {},
  "leagueLeaders": {},
  "transactions": [],
  "injuries": [],
  "historicalItems": [],
  "generationMetadata": {}
}
```

The renderer must consume this edition object and should not contain business logic for selecting stories or calculating baseball statistics.

21. Separation of Responsibilities

The implementation should be separated into the following components.

21.1 Data Collectors

Responsibilities:

- Retrieve raw data
- Handle authentication
- Handle rate limits
- Retry failed requests
- Save raw responses
- Record provider timestamps

Collectors should not generate HTML.

21.2 Normalization Layer

Responsibilities:

- Convert provider-specific responses into internal models
- Standardize team IDs
- Standardize player IDs
- Standardize game status values
- Normalize date and time formats
- Deduplicate records
- Reconcile conflicting data

21.3 Statistical Processor

Responsibilities:

- Calculate or select standings
- Rank league leaders
- Determine qualification
- Calculate derived categories
- Identify notable performances
- Detect milestones
- Generate editorial ranking signals

21.4 Editorial Generator

Responsibilities:

- Select the lead story
- Select secondary stories
- Generate headlines and decks
- Generate game recaps
- Generate league summaries
- Produce structured editorial content
- Attach source facts to each generated story

21.5 Validation Layer

Responsibilities:

- Confirm scores match structured data
- Confirm player-team relationships
- Confirm stat values
- Detect unsupported claims
- Detect duplicate text
- Reject malformed content
- Reject empty required sections

- Check AI output against source facts

21.6 HTML Renderer

Responsibilities:

- Convert edition JSON into HTML
- Apply templates
- Escape unsafe text
- Generate tables
- Generate metadata
- Add structured data
- Add print styles
- Produce deterministic output

21.7 Publisher

Responsibilities:

- Validate generated files
 - Publish assets
 - Atomically replace `index.html`
 - Preserve last known good edition
 - Record publication status
 - Purge CDN cache when necessary
 - Roll back failed publications
-

22. AI Content Requirements

AI-generated content must be grounded entirely in supplied sports data.

The model must not be asked to retrieve scores or statistics from memory.

Every generation prompt should include:

- Relevant structured game data
- Player statistics
- Standings context
- Known injuries
- Transaction data

- Required article format
- Maximum length
- Prohibited unsupported claims

AI output should use structured JSON rather than free-form text where possible.

Example:

```
{
  "headline": "",
  "deck": "",
  "paragraphs": [],
  "factsUsed": [
    {
      "factId": "",
      "description": ""
    }
  ]
}
```

22.1 AI Prohibitions

The editorial generator must not:

- Invent quotes
- Invent injuries
- Invent transactions
- Invent betting lines
- Invent attendance
- Invent weather
- Invent player statistics
- Invent standings implications
- Claim a record without verified data
- Describe a play not present in the source data
- Impersonate a living or deceased sportswriter

The system may use broad tone descriptors such as:

- Traditional newspaper
- Dry humor
- Analytical
- Conversational
- Old-fashioned baseball prose

It should not explicitly imitate a named writer or broadcaster.

23. Editorial Selection Logic

Lead-story selection should be deterministic enough to explain and test.

A configurable scoring model should consider:

- Playoff importance
- Historic significance
- Game leverage
- Market or national relevance
- Individual performance
- Comeback size
- Walk-off status
- Extra innings
- Rivalry significance
- No-hitter status
- Record or milestone
- Trade significance
- Injury significance

Example conceptual scoring:

```
lead_score =  
    playoff_weight  
    + historic_weight  
    + performance_weight  
    + game_drama_weight  
    + national_interest_weight  
    + recency_weight
```

The precise values should be configuration-driven.

Manual editorial overrides should be supported through an optional configuration file.

Example:

```
lead_story_game_id: 123456  
featured_player_ids:  
    - 592450
```



```
suppress_story_ids:  
  - injury-987
```

24. HTML and CSS Requirements

The visual design should use:

- Serif typography
- Strong masthead
- Multi-column editorial layouts
- Thin and double horizontal rules
- Restrained color palette
- Newsprint-inspired background
- Compact tables
- Drop caps
- Small uppercase labels
- Black-and-white or muted imagery

Avoid:

- Large rounded cards
- App-style floating controls
- Excessive gradients
- Bright team-color dominance
- Infinite scrolling patterns
- Large empty areas
- Heavy animation
- Horizontal carousels for core content

24.1 Responsive Behavior

Desktop:

- Multi-column newspaper layout
- Sidebars
- Dense standings and statistics
- Large masthead

Tablet:

- Two-column layouts where practical
- Reduced table columns

- Maintained newspaper styling

Mobile:

- Single-column content flow
- Readable tables
- Horizontal scrolling only where unavoidable
- Sticky section navigation optional
- No text smaller than a readable minimum
- Touch targets sized appropriately

Mobile design should be responsive browser design, not an application interface.

25. Print Requirements

The page must include a dedicated print stylesheet.

Printing should:

- Remove interactive controls
- Remove unnecessary navigation
- Use black text on a light background
- Avoid splitting tables unnecessarily
- Avoid orphaned headlines
- Preserve section headings
- Display URLs only when useful
- Format cleanly for standard letter-size paper
- Support browser “Save as PDF”

Use:

```
@media print {  
}
```

The printed version should resemble a newspaper supplement.

26. Performance Requirements

The page should be optimized for static delivery.

Targets:

- Main HTML under 500 KB where practical
- Total initial page weight under 2 MB without large photography
- Largest Contentful Paint under 2.5 seconds on a typical broadband connection
- Minimal render-blocking JavaScript
- No large frontend framework required
- JavaScript bundle under 100 KB compressed where practical
- CSS under 100 KB compressed
- Images optimized as WebP or AVIF with fallbacks as needed

Because the page is static, it should remain readable even during backend or data-provider outages.

27. Caching Requirements

The page should use CDN and browser caching appropriately.

Recommended behavior:

For `index.html`:

```
Cache-Control: public, max-age=60, must-revalidate
```

For versioned assets:

```
Cache-Control: public, max-age=31536000, immutable
```

If assets are not versioned, shorter cache durations must be used.

Each publication should include a unique edition identifier in the HTML.

Example:

```
<meta name="sportzballz-edition-id" content="2026-07-13-0600"
```

The page may display a visible refresh link, but automatic aggressive browser refreshing is not recommended.

28. Atomic Publication

A partially generated page must never replace the current edition.

Required publication process:

1. Generate into a temporary directory.
2. Validate all required files.
3. Check HTML syntax.
4. Check internal links and assets.
5. Confirm required sections exist.
6. Confirm publication timestamp.
7. Copy or upload assets first.
8. Move the new HTML into place atomically.
9. Verify the public URL.
10. Record success.
11. Purge or refresh CDN cache if required.

Example filesystem flow:

```
/tmp/daily-sports-page-build-20260713-0600/  
    ↓  
validation  
    ↓  
/var/www/sportzballz.io/daily-sports-page/index.html.tmp  
    ↓  
atomic rename  
    ↓  
/var/www/sportzballz.io/daily-sports-page/index.html
```

The previous successful edition must remain available for rollback.

29. Failure and Degraded Modes

The publication pipeline must handle incomplete source data.

29.1 Data Provider Failure

If a provider is unavailable:

- Retry with exponential backoff
- Use cached data if it is within an acceptable age
- Mark affected data with a freshness timestamp
- Omit unavailable optional sections
- Do not invent missing data

- Continue publication when core requirements are satisfied

29.2 AI Provider Failure

If AI generation fails:

- Use deterministic templates
- Publish scores, standings, schedules, and statistics
- Reuse only content generated for the same game state when safe
- Do not block the entire edition

29.3 Rendering Failure

If HTML rendering or validation fails:

- Do not publish
- Keep the current edition live
- Save failure artifacts
- Log the error
- Send an alert

29.4 Stale Edition

If no successful publication occurs within the expected window, the current page should show:

Latest edition generated at [timestamp].
Some information may be delayed.

30. Data Freshness Rules

Each section should carry internal freshness metadata.

Example:

```
{
  "scoresUpdatedAt": "",
  "standingsUpdatedAt": "",
  "leadersUpdatedAt": "",
  "transactionsUpdatedAt": "",
  "injuriesUpdatedAt": ""
}
```

The visible page should show the overall data cutoff.

Sections using older data may show a small update label.

The publisher should define maximum acceptable ages.

Recommended defaults:

- Live scores: 5 minutes
- Scheduled games: 30 minutes
- Standings: 30 minutes
- League leaders: 6 hours
- Transactions: 30 minutes
- Injuries: 2 hours
- Historical data: 30 days

These values must be configurable.

31. Search Engine Requirements

Each edition should include:

- Unique page title
- Meta description
- Canonical URL
- Open Graph metadata
- Social preview image
- Structured article metadata where applicable
- Semantic headings
- Crawlable text
- Descriptive anchor links

Example:

```
<title>MLB Daily Sports Page — July 13, 2026 | SportzBallz</title>
<meta
  name="description"
  content="Today's MLB scores, standings, league leaders, ga
>
<link
  rel="canonical"
  href="https://sportzballz.io/daily-sports-page/index.html"
>
```

If archived editions are created, each archive page should have its own canonical URL.

32. Social Sharing Requirements

The page should support sharing:

- Entire edition
- Individual section
- Individual story
- Individual game recap
- League-leader category

Each shareable item should have:

- Stable anchor or static URL
- Headline
- Description
- Optional generated preview image
- Open Graph metadata where separate pages exist

For the MVP, anchor-based sharing within `index.html` is sufficient.

33. Accessibility Requirements

The page should target WCAG 2.2 AA.

Requirements include:

- Semantic HTML
- Logical heading hierarchy
- Keyboard navigation
- Visible focus states
- Adequate contrast
- Text resizing support
- Table headings and captions
- Accessible names for controls
- Alternative text for meaningful images
- Reduced-motion support
- No essential information conveyed only through color

- Skip-to-content link

Interactive leader tabs must use appropriate ARIA attributes or degrade to visible sections.

34. Browser Support

Required browser support:

- Current Chrome
- Current Edge
- Current Firefox
- Current Safari
- Mobile Safari
- Chrome for Android

The page should remain readable in older browsers even if enhancements are unavailable.

No browser-specific functionality should be essential to reading the edition.

35. Security Requirements

Because the page is generated from external data and AI output:

- Escape all HTML content by default
- Sanitize any permitted markup
- Prevent script injection
- Do not insert untrusted raw HTML
- Use a restrictive Content Security Policy
- Avoid third-party scripts where practical
- Pin external asset versions
- Use HTTPS only
- Do not expose provider API keys
- Do not include private generation metadata in public HTML
- Validate all URLs used in links and images

Recommended headers:

Content-Security-Policy
X-Content-Type-Options: nosniff
Referrer-Policy
Permissions-Policy
Strict-Transport-Security

36. Observability and Logging

Each generation run should record:

- Run ID
- Edition ID
- Start time
- Completion time
- Data collection duration
- AI generation duration
- Render duration
- Publication duration
- Providers used
- Provider failures
- Number of games
- Number of stories
- Number of leader categories
- Validation results
- Published URL
- Final status

Statuses:

started
collecting
normalizing
generating
validating
rendering
publishing
published
failed
published_degraded

Metrics should be suitable for automated alerting.

37. Testing Requirements

37.1 Unit Tests

Cover:

- Team normalization
- Player normalization
- Date conversion
- Game-state mapping
- Standings calculations
- Top-10 ranking
- Qualification rules
- Lead-story scoring
- HTML escaping
- Edition metadata

37.2 Integration Tests

Cover:

- Provider response to normalized model
- Normalized model to edition JSON
- Edition JSON to HTML
- AI output validation
- Publication to staging directory
- Archive creation
- Rollback behavior

37.3 Snapshot Tests

Use representative fixtures for:

- Full 15-game slate
- Partial slate
- Opening Day
- All-Star break
- Postseason
- Doubleheader
- Postponements
- Extra-inning games
- No-hitter

- Trade deadline
- No scheduled games

Generated HTML should be snapshot-tested.

37.4 Visual Regression Tests

Capture desktop, tablet, and mobile screenshots.

Required viewport examples:

1440 × 1200

1024 × 1366

768 × 1024

390 × 844

Visual tests should check:

- Overflow
- Broken columns
- Table readability
- Missing assets
- Headline wrapping
- Print layout
- Dark or evening theme, if retained

37.5 Data Accuracy Tests

Automatically compare rendered values against edition JSON for:

- Scores
- Team names
- Standings
- League leaders
- Pitcher decisions
- Game status
- Dates and times

38. Recommended Repository Structure

daily-sports-page/

└─ README.md

└─ pyproject.toml

```
├─ package.json
├─ config/
│   ├─ settings.yaml
│   ├─ editorial.yaml
│   ├─ schedules.yaml
│   └─ teams.yaml
├─ src/
│   ├─ collectors/
│   ├─ normalization/
│   ├─ statistics/
│   ├─ editorial/
│   ├─ validation/
│   ├─ rendering/
│   ├─ publishing/
│   └─ models/
├─ templates/
│   ├─ index.html.j2
│   ├─ sections/
│   └─ games/
├─ static/
│   ├─ css/
│   ├─ js/
│   └─ images/
├─ prompts/
│   ├─ lead-story.md
│   ├─ game-recap.md
│   ├─ league-roundup.md
│   └─ headline.md
├─ schemas/
│   ├─ edition.schema.json
│   └─ editorial.schema.json
├─ tests/
│   ├─ unit/
│   ├─ integration/
│   ├─ snapshots/
│   ├─ visual/
│   └─ fixtures/
├─ build/
├─ scripts/
│   ├─ generate-edition.sh
│   ├─ publish-edition.sh
│   └─ validate-edition.sh
```

```
└─ .github/  
  └─ workflows/
```

The specific programming language may vary, but these responsibilities should remain separated.

39. Recommended Technical Stack

The implementation should favor simple, reliable static generation.

Recommended backend and generation stack:

```
Python 3.12+  
Pydantic  
Jinja2  
HTTPX  
Tenacity  
Beautiful Soup or equivalent for validation only  
JSON Schema  
Pytest  
Playwright  
Ruff  
MyPy
```

Optional supporting tools:

```
Node.js  
PostCSS  
Lightning CSS  
esbuild  
Prettier  
HTMLHint
```

A frontend framework is not required.

The preferred browser implementation is:

```
Semantic HTML  
Modern CSS  
Minimal vanilla JavaScript
```

40. Agent Workstream Assignments

Coding agents should be assigned clear, non-overlapping responsibilities.

Agent 1: Domain Model and Schemas

Deliverables:

- Edition schema
- Game schema
- Player schema
- Team schema
- Standings schema
- League-leader schema
- Transaction schema
- Injury schema
- Validation tests

Agent 2: Sports Data Integration

Deliverables:

- MLB data collector
- Retry handling
- Raw-response caching
- Provider timestamp tracking
- Team and player identifier mapping
- Fixture generation

Agent 3: Normalization and Statistics

Deliverables:

- Normalized MLB models
- Standings output
- League-leader ranking
- Qualification rules
- Derived game metrics
- Notable-performance detection

Agent 4: Editorial Pipeline

Deliverables:

- Lead-story ranking

- Story selection
- Prompt templates
- Structured AI output
- Fact-reference attachment
- Deterministic fallback copy

Agent 5: Fact Validation

Deliverables:

- Score validation
- Player-stat validation
- Team validation
- Unsupported-claim detection
- Duplicate-content detection
- Publication blocking rules

Agent 6: HTML and CSS Renderer

Deliverables:

- Jinja or equivalent templates
- Newspaper design system
- Responsive layout
- League-leader interactions
- Print stylesheet
- Semantic HTML
- Accessibility support

Agent 7: Publishing Pipeline

Deliverables:

- Build command
- Validation command
- Atomic publication
- Archive support
- CDN purge hook
- Rollback support
- Staging and production configuration

Agent 8: QA and Visual Testing

Deliverables:

- Unit-test suite
- Integration suite
- HTML snapshots
- Playwright browser tests
- Visual regression tests
- Accessibility checks
- Broken-link checks

Agent 9: Observability

Deliverables:

- Structured logs
 - Run-state tracking
 - Failure alerts
 - Publication metrics
 - Last-known-good tracking
 - Edition health report
-

41. Command-Line Interface

The generator should expose a simple command-line interface.

Recommended commands:

```
daily-sports-page collect
daily-sports-page normalize
daily-sports-page generate
daily-sports-page validate
daily-sports-page render
daily-sports-page publish
daily-sports-page run
```

Example complete run:

```
daily-sports-page run \
  --edition morning \
  --date 2026-07-13 \
  --timezone America/New_York \
  --publish
```

Dry-run mode:


```
daily-sports-page run --dry-run
```

Rebuild from saved data:

```
daily-sports-page render \  
  --edition-json build/2026-07-13-0600/edition.json
```

This separation will let agents test rendering without repeatedly calling sports or AI APIs.

42. Configuration Requirements

The system should be configurable using environment variables and configuration files.

Configurable values:

- Publication schedule
- Time zone
- Output path
- Public base URL
- Data-provider credentials
- AI-provider credentials
- Edition section order
- Enabled sections
- Maximum story lengths
- Lead-story weights
- Data freshness limits
- Archive behavior
- CDN settings
- Theme and publication name
- Number of league leaders
- Betting content enabled or disabled
- Image generation enabled or disabled

Secrets must come from environment variables or a secrets manager, not committed files.

43. MVP Scope

The first production release should contain:

- Static `index.html`
- MLB-wide morning and evening editions
- Masthead and timestamp
- League-wide lead story
- Yesterday's scores
- Current and upcoming games
- Compact game recaps
- Full division standings
- Wild-card standings
- Top 10 league leaders
- Transactions
- Significant injuries
- Responsive newspaper layout
- Print stylesheet
- Edition JSON
- Automated generation
- Atomic publication
- Last-known-good rollback
- Basic monitoring
- Data and content validation

44. Items Explicitly Outside MVP Scope

The following are not required:

- Native mobile applications
- React Native
- Flutter
- User registration
- User login
- Personalized team editions
- User preferences
- Push notifications
- Live WebSocket updates
- Runtime sports API calls from the browser
- Runtime AI calls from the browser
- Comments
- Social network features

- Fantasy roster management
 - Sportsbook account integration
 - Subscription billing
 - Full-content management system
 - Infinite scrolling
 - Complex client-side application state
-

45. Acceptance Criteria

The MVP is complete when:

1. An automated command generates a valid static MLB edition.
2. The output is published to:

`https://sportzballz.io/daily-sports-page/index.html`

3. The page contains all scheduled and completed MLB games available at generation time.
 4. The page contains current standings.
 5. The page contains top 10 players for all required leader categories.
 6. The lead story and recaps are grounded in structured data.
 7. Every score and displayed statistic can be traced to edition JSON.
 8. The page remains readable without JavaScript.
 9. The page works on current desktop and mobile browsers.
 10. The page prints cleanly.
 11. A failed run does not replace the last successful edition.
 12. The publication timestamp and data cutoff are visible.
 13. The generation pipeline records a clear success or failure state.
 14. The page contains no Philadelphia-first personalization.
 15. The page presents MLB as a complete national league-wide product.
-

46. Definition of Done for Each Publication

A publication run is considered successful only when:

- Data collection completed or an approved cached-data fallback was used.
 - Edition JSON passed schema validation.
 - Required sections are populated.
 - All scores and statistics passed validation.
 - Editorial content passed factual checks.
 - HTML rendered successfully.
 - HTML passed structural validation.
 - Referenced assets exist.
 - Accessibility smoke tests passed.
 - The temporary build was published atomically.
 - The public URL returned a successful response.
 - The published edition ID matched the generated edition ID.
 - The run was recorded as `published` or `published_degraded`.
-

47. Guiding Implementation Principle

The product is not a mobile application and should not be built like one.

It is an automatically generated digital newspaper.

All implementation decisions should support this model:

Collect current MLB data, create a complete editorial edition, render it as durable static HTML, and publish it reliably several times each day.

This version gives the coding agents a much tighter target: a static-generation pipeline, deterministic edition model, minimal browser JavaScript, and atomic deployment to the SportzBallz URL.